
ROBOFEST 6.0 | AUTONOMOUS UNDERWATER VEHICLE

SIMULATION & MAPPING

Simulation Environment, System Architecture,
Mission Navigation & Mapping Pipeline



ADD IMAGE – AUV Hero Shot

Simulation or real photo of the AUV, used as the cover
image.

save as: docs/photos/00-auv-hero.jpg

Competition: RoboFest 6.0

Document: Simulation & Mapping Technical Report

Simulator: Gazebo Harmonic + ROS 2 Jazzy

Date: August 24, 2026

*Object detection (YOLO training, dataset, metrics) is covered in a separate
Object Detection Documentation, to be combined with this report at final submission.*

Contents

1	Executive Summary	2
2	Mission Overview	2
3	Simulation Environment	2
3.1	Gazebo World	2
3.2	AUV Vehicle Model	3
3.3	Sensor Suite	4
3.4	Motion & Physics	4
4	System Architecture	4
4.1	Workspace Layout	4
4.2	ROS 2 – Gazebo Bridge	4
4.3	Node Graph	5
5	Mission Navigation	5
5.1	Gate Navigation	5
5.2	Green Zone & Blue Bin	6
6	Mapping & Localisation	6
6.1	Odometry-Based Localisation	6
6.2	Trajectory (Trail) Mapping	6
6.3	Sim-to-Real Localisation	7
7	Testing & Validation	7
7.1	Nominal Mission Run	7
7.2	Live Monitoring Commands	8
8	Launch & Operation	8
9	Known Limitations & Next Steps	8
10	Conclusion	9

1 Executive Summary

This report documents the simulation environment, system architecture, navigation behaviour, and mapping pipeline built for an Autonomous Underwater Vehicle (AUV) competing in **RoboFest 6.0**. The full software stack runs on **ROS 2 Jazzy** inside a **Gazebo Harmonic** simulated pool, and is designed so that moving to real hardware requires only a configuration change, not a code change.

At a Glance

- Full three-phase mission: gate crossing (round trip) → green zone acquisition → blue bin hold.
- Ground-truth odometry at 50 Hz, with a trail-mapping node that records and visualises the AUV's trajectory live in RViz2.
- Every tunable value lives in `sim_params.yaml`; no source-code changes are needed when moving to real hardware.
- Single-command launch: `ros2 launch auv_bringup sim_mission.launch.py`.

Scope of this document. This report covers the *simulation environment, system integration, mission navigation, and mapping/localisation*. Object detection (YOLO model training, dataset preparation, HSV tuning, detection accuracy) is developed and documented separately by the perception team; the two reports will be combined into a single submission package.

2 Mission Overview

The AUV executes a single continuous, fully autonomous mission in three phases:

1. **Gate Navigation** – search for, approach, align with, and cross an underwater gate, then repeat the crossing on the return leg.
2. **Green Zone Acquisition** – locate a green target mat on the pool floor and hold position above it.
3. **Blue Bin Hold** – locate a blue bin on the mat and hold position above it to close out the mission.



Figure 1: Mission phase flow: gate round trip, then green zone, then blue bin hold.

3 Simulation Environment

3.1 Gazebo World

The mission is simulated in Gazebo Harmonic, chosen for native ROS 2 integration, full 6-DOF rigid-body physics, and high-fidelity stereo camera simulation. The world file (`camera_test.sdf`) places the gate, green mat, blue bin, and AUV spawn point at fixed coordinates, with zero world gravity – the AUV holds depth through its motion plugin rather than buoyancy in this phase.

Table 1: Key world elements

Element	Position	Description
Gate	$(0, 12, -4.5)$	Vertical poles + crossbar
Green mat	$(10, 22, -5.0)$	Target zone on pool floor
Blue bin	$(10, 22, -4.75)$	Sits on the green mat
AUV spawn	$(-5, 0, z_d)$	Configurable via launch arguments



ADD IMAGE – Gate in Gazebo

Overhead or angled Gazebo screenshot showing the gate structure clearly.

save as: docs/photos/01-gate-world.png



ADD IMAGE – Green Mat & Blue Bin

Gazebo screenshot of the green target mat with the blue bin resting on it.

save as: docs/photos/02-mat-bin-world.png

3.2 AUV Vehicle Model

The vehicle model (auv_box) is built from the design team’s CAD export and carries realistic mass and inertia properties.

Table 2: AUV physical properties

Property	Value	Notes
Bounding box	$0.92 \times 0.83 \times 0.58 \text{ m}$	Collision geometry
Mass	11.74 kg	Design team CAD value
Visual mesh	base_link.STL	SolidWorks export



ADD IMAGE – AUV Vehicle

Gazebo screenshot of the AUV model, and/or a photo of the real vehicle.

save as: docs/photos/03-auv-model.png

3.3 Sensor Suite

The AUV carries three cameras modelled after the OAK-D Lite: a front stereo pair (0.075 m baseline) for depth estimation and gate localisation, and a downward-facing bottom camera for close-range zone centring.

Table 3: Camera specifications

Parameter	Value	Notes
Resolution	640×480 px	All three cameras
Horizontal FOV	$\approx 73^\circ$	OAK-D Lite spec
Update rate	15 Hz	All cameras
Range	0.2 – 19.1 m	Near / far clip planes
Stereo baseline	0.075 m	Front pair, centre-to-centre

3.4 Motion & Physics

In this phase the AUV is driven by a kinematic velocity-control plugin (perfect velocity tracking, no thruster tuning required), which decouples navigation development from thruster dynamics. Hydrodynamic drag coefficients (Fossen model, BlueROV2 basis) are pre-configured in the world file and activate automatically once thruster allocation is implemented, without any change to the navigation logic.

4 System Architecture

4.1 Workspace Layout

```
~/auv_ws/
|-- src/
|   |-- auv_planner/      # Mission FSMs: gate + green navigators
|   |-- auv_vision/       # Perception nodes (see companion doc)
|   |-- auv_telemetry/    # CSV logger, dashboard, trail mapper
|   |-- auv_description/  # SDF model, meshes, launch, RViz config
|   '-- auv_bringup/      # Top-level launch + sim_params.yaml
|-- camera_test.sdf       # Gazebo world file
'-- mission_logs/         # Auto-created CSV and trail logs
```

Listing 1: AUV workspace structure (active packages)

4.2 ROS 2 – Gazebo Bridge

All Gazebo topics are bridged into the /auv/ ROS 2 namespace, so downstream nodes never reference Gazebo-specific topic names directly.

Table 4: Core bridged topics

Topic	Type	Dir.
.../left/image_raw	Image	GZ→ROS
.../right/image_raw	Image	GZ→ROS
.../bottom/image_raw	Image	GZ→ROS
odometry → /auv/odom	Odometry	GZ→ROS
cmd_vel	Twist	ROS→GZ

4.3 Node Graph

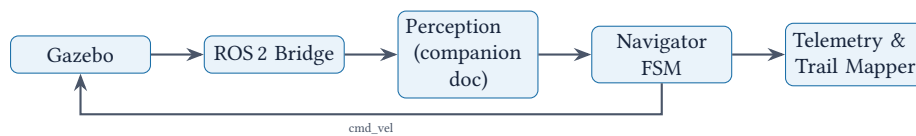


Figure 2: High-level data flow. Perception detail is covered in the companion document.

5 Mission Navigation

The navigator nodes consume 3D object positions published by the perception pipeline (over well-defined /auv/ topics) and convert them into velocity commands. This section covers only that navigation and control behaviour – the detection algorithms themselves are documented separately.

5.1 Gate Navigation

The gate is crossed on both the forward and return legs using a search → track → align → cross sequence, repeated in reverse for the return trip. The gate’s position is stored in the world-fixed odometry frame (not the camera frame), which lets the AUV keep navigating toward it even during the brief periods when it is out of camera view.

Table 5: Gate navigation sequence

Stage	Behaviour
Search	Sweep laterally until the gate is detected
Track	Drive toward the gate, refining its 3D position
Align	Rotate to face the gate opening squarely
Cross	Drive straight through, then repeat in reverse for the return leg



ADD IMAGE – AUV Crossing the Gate

Gazebo screenshot of the AUV mid-crossing through the gate opening.

save as: docs/photos/04-gate-crossing.png

5.2 Green Zone & Blue Bin

After the gate round trip, the navigator switches to a search → approach → centre sequence over the green mat, then repeats a shorter approach over the blue bin before holding position to close out the mission.

Robustness Notes

- Stored target coordinates are frozen once the AUV is very close, since bounding-box centres become unreliable at short range.
- Any single position update that jumps implausibly far is discarded, protecting the stored target from a single bad detection frame.
- If detections go briefly stale, the navigator falls back to odometry dead-reckoning toward the last known target position.

6 Mapping & Localisation

6.1 Odometry-Based Localisation

Gazebo's odometry plugin publishes ground-truth 6-DOF pose at 50 Hz on `/auv/odom`, which is the single source of position and heading used throughout the stack – by the navigators, the telemetry logger, and the trail mapper.

Table 6: Odometry topic

Property	Value
Topic	<code>/auv/odom</code>
Rate	50 Hz
Frame	map (world-fixed)
Data	Position, orientation, and velocity

6.2 Trajectory (Trail) Mapping

A dedicated trail-mapper node records the AUV's path as a `nav_msgs/Path`, adding a new point only once the AUV has moved at least 0.10 m, and publishes it at 1 Hz for live visualisation.

Table 7: Trail mapper output

Property	Value
Min. point spacing	0.10 m
Publish rate	1.0 Hz
Output	/auv/trail_map (RViz2) + CSV log



ADD IMAGE – RViz2 Trajectory Map

RViz2 screenshot showing the recorded trail (green line) from spawn through the gate to the green mat, with the AUV's current pose marker.

save as: docs/photos/05-rviz-trajectory.png

A pre-configured RViz2 layout renders this live: a pose arrow at the AUV's current position and a growing trail line for the full trajectory, on a world-fixed map frame with a dark background for underwater readability.

6.3 Sim-to-Real Localisation

Table 8: Localisation: simulation vs. real hardware

Component	Simulation	Real Hardware
Position	Gazebo odometry (ground truth)	EKF fusing DVL + IMU
Heading	Gazebo quaternion (perfect)	IMU magnetometer
Mapping pipeline	Identical – same trail mapper and RViz2 config	

7 Testing & Validation

7.1 Nominal Mission Run

A full simulation run is validated end-to-end against a fixed scenario: AUV spawned at $(-5, 0, -4.238)$ facing the gate, gate at $(0, 12, -4.5)$, green mat at $(10, 22, -5.0)$.

1. Gate round trip: search, track, align, cross, then repeat in reverse.
2. Hand-off to the green navigator on gate completion.
3. Green mat search, approach, and centring.
4. Blue bin search, approach, and position hold.
5. Mission marked complete; telemetry and trail logs finalised.

An adversarial variant spawns the AUV off-axis (facing *away* from the gate, with a lateral offset), forcing a wide search sweep before the gate enters camera view – this exercises the odom-frame target tracking and outlier rejection described in Section 5 under realistic worst-case conditions.

7.2 Live Monitoring Commands

```
# Watch mission phase transitions live
ros2 topic echo /auv/mission_state

# Watch AUV pose and velocity
ros2 topic echo /auv/odom

# Record a full mission for later playback and analysis
ros2 bag record -a -o ~/auv_ws/mission_logs/run_bag/
```

Listing 2: Useful commands during a simulation run

8 Launch & Operation

```
source ~/auv_ws/install/setup.bash
ros2 launch auv_bringup sim_mission.launch.py

# Optional: custom spawn position
ros2 launch auv_bringup sim_mission.launch.py \
    spawn_x:=-5.0 spawn_y:=0.0 spawn_z:=-4.238 spawn_yaw:=0.0
```

Listing 3: Full mission launch

A single launch file starts Gazebo, spawns the AUV, brings up the ROS 2 bridge, and starts all mission nodes in the correct order automatically. Mission telemetry (CSV logs and trail data) is written to `mission_logs/` for every run.

9 Known Limitations & Next Steps

The simulation stack is deliberately built to de-risk navigation and mapping before hardware is involved; a few gaps are already understood and planned for.

Table 9: Current limitations and planned resolutions

Limitation	Impact	Planned Resolution
Kinematic velocity control	No real hydrodynamic response yet	Thruster allocation package (Phase 2)
Ground-truth odometry	Won't be available on real hardware	EKF fusing DVL + IMU
Planar (XY) navigation	No active depth control	Z-axis depth controller
Fixed world layout	Tuned to a single gate pose	Generalise to arbitrary gate placement

None of these require changes to the mapping pipeline itself: the trail mapper and RViz2 visualisation already consume odometry through the same `/auv/odom` topic that the real EKF will publish to.

10 Conclusion

This report has covered the simulation environment, system architecture, mission navigation behaviour, and mapping/localisation pipeline for the RoboFest 6.0 AUV. The environment models the gate, green zone, and blue bin mission with a realistic vehicle model and camera suite; navigation executes a robust search-track-align-cross sequence with odom-frame target tracking; and the mapping pipeline provides full trajectory visibility through ground-truth odometry, trail recording, and live RViz2 visualisation. All parameters are externalised to configuration files, so the transition to real hardware requires no change to this logic.

Companion Document

Object detection – model training, dataset preparation, and detection accuracy – is documented separately by the perception team. The two documents will be combined into a single submission package.